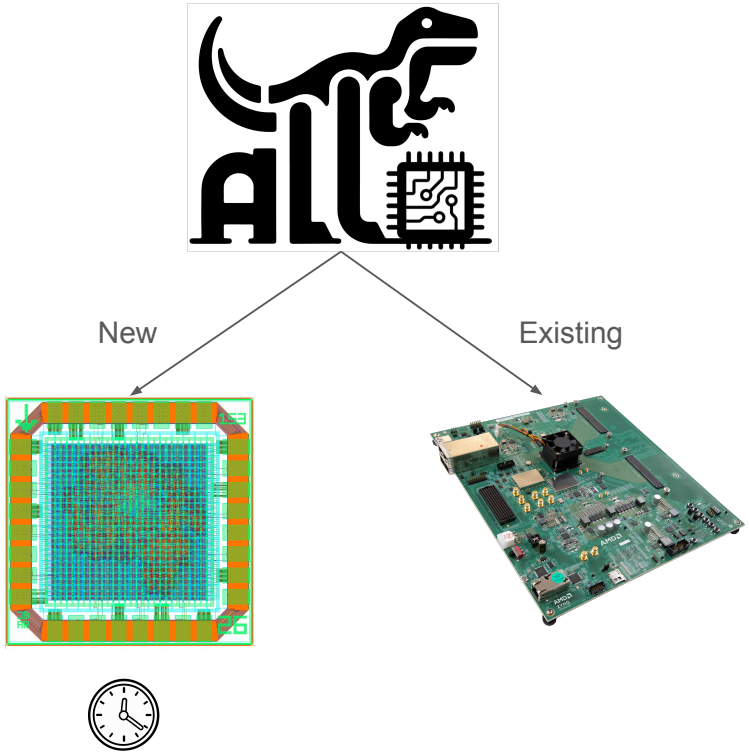
AAAH: Allo-Assisted ASIC Harness

Julian Bushlow



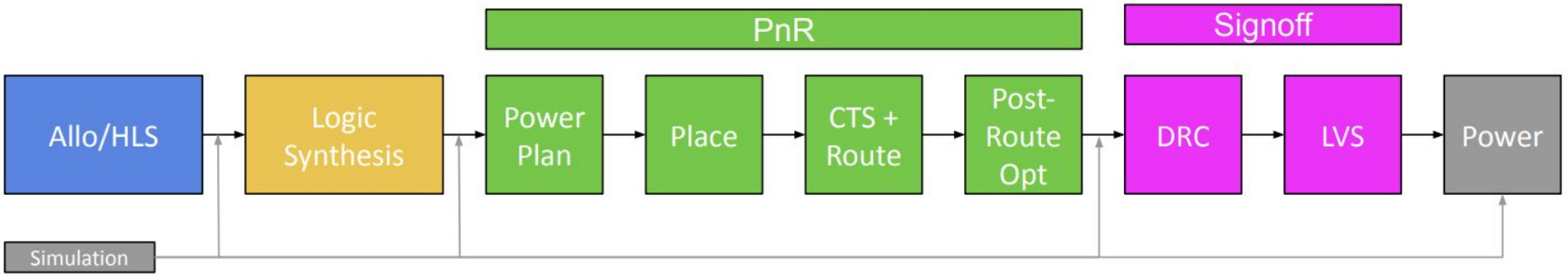
New Allo → ASIC Path… Too Slow!

- Previously could only target FPGAs or ASIC backends ended at RTL level
- Built RTL → GDS flow… works but too slowly
  - Especially for verification during design space exploration
- Idea: fast hierarchical flow using high level module/connection information
  - Automate tasks as much as possible



16x16 int32 GEMM run: 7 hours (ASIC) 24 minutes (Vitis)

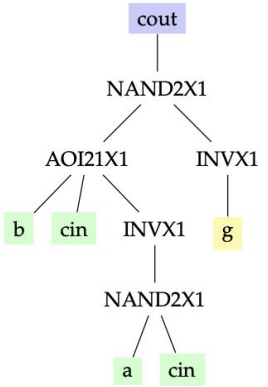
Basic ASIC Flow

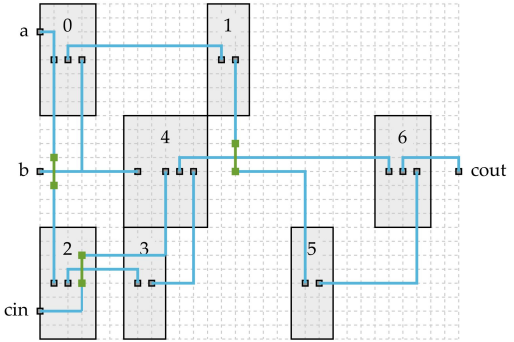


From 16x16 int32 GEMM run (flat)

Basic ASIC - Synthesis + Place

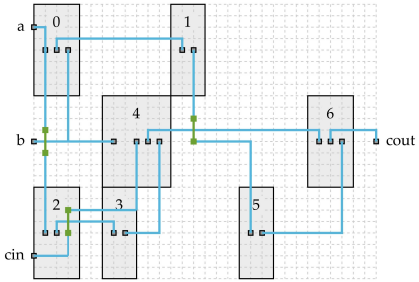
- Synthesis - RTL → logic netlist → techmap
  - Parse RTL into logical operations
  - Map onto available technology (stdcells)
  - Similar to FPGA synthesis, but available tech is a stdcell library, not LUTs/DSPs
- Place - tile stdcells in 2D plane
  - Large designs - millions of stdcells
  - Can optimize every individual position, scales poorly with design size

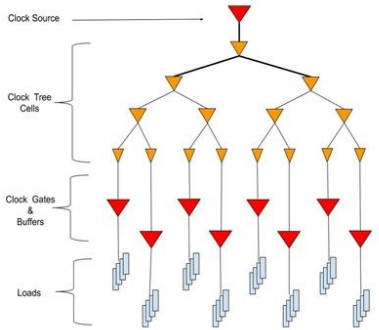


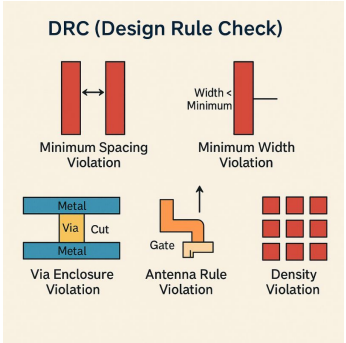


Basic ASIC - Route + Clock Tree + Design Rule Checking

- Route - connect stdcells with wires
- Clock Tree Synthesis - special routing
  - Deliver clock signals simultaneously with tree structure
- Design Rule Checking - make sure stdcells/wires don’t collide, etc
- All scale poorly with design size!
  - More nets to track
  - Higher fanout in clock tree
  - More polygons to check

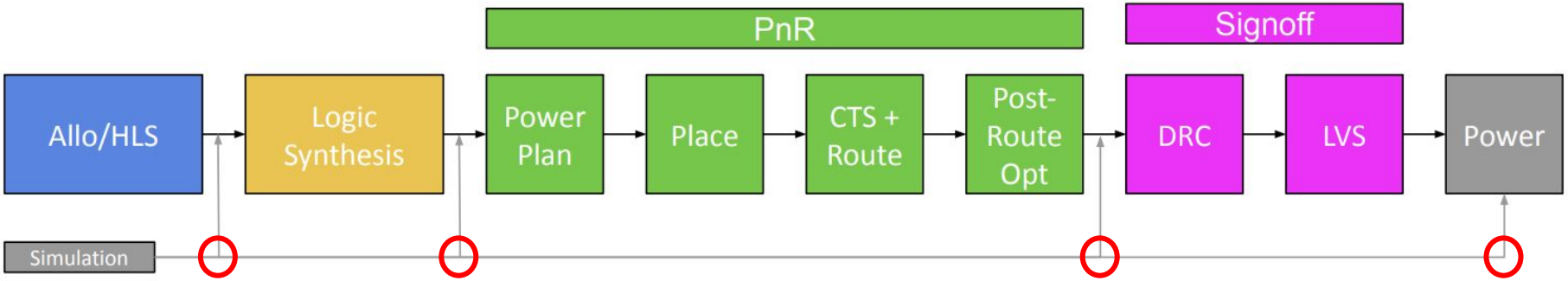






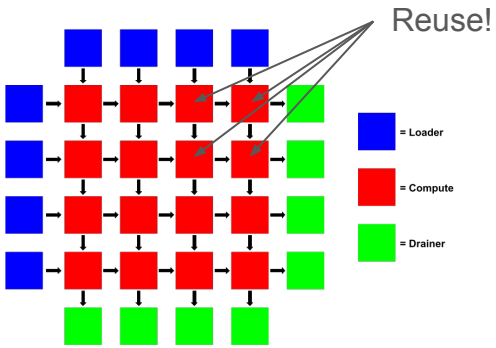
Basic ASIC - Simulation

- Required for verification + power
- Need a Verilog testbench and a workload



Redundancies/Limitations of RTL Representation

- Spatial architectures have frequent module reuse (SPMW)
  - ASIC flows will optimize globally even though same logic is present many times
- Spatial architectures designed with placement in mind
  - RTL doesn't encode spatial information
  - Allo does!




```
module node_1_0_run_v2270_Pop_mioi_v2270_Pop_mio_wait_dp (
  clk, rst, v2270_Pop_mioi_oswt, v2270_Pop_mioi_wen_comp, v2
      v2270_Pop_mioi_biwt, v2270_Pop_mioi_bdwt, v2270_Pop_mi
);
  input clk;
  input rst;
  input v2270_Pop_mioi_oswt;
  output v2270_Pop_mioi_wen_comp;
  output [16:0] v2270_Pop_mioi_idat_mxwt;
  input v2270_Pop_mioi_biwt;
  input v2270_Pop_mioi_bdwt;
  output v2270_Pop_mioi_bcwt;
  reg v2270_Pop_mioi_bcwt;
  input [16:0] v2270_Pop_mioi_idat;
```

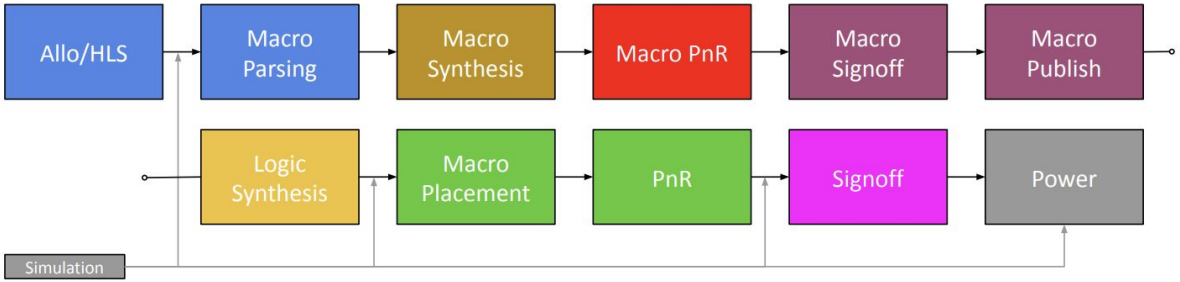
only know connections exist…

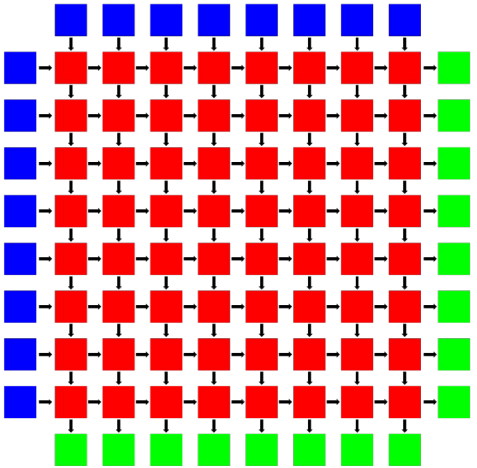
```
@df.kernel(mapping=[M, N])
def node():
    i, j = df.get_pid()
    # ---- PE core state: verbatim
    irf: int32[IRF_DEPTH] = 0
    drf: Ty[DRF_DEPTH] = 0
    drf_full: int32[DRF_DEPTH] = 0
    dsmask: int32 = 0
```

know *where* to connect

Key Idea: Two-Stage Hierarchical ASIC Flow

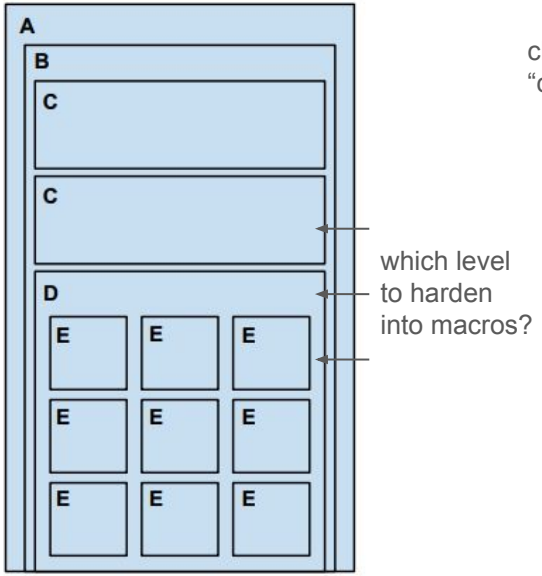
- Stage 1: Optimize small, reusable sections of design, save as macros
- Stage 2: Tile macros across chip
- Great for designs with high module reuse
- Can parallelize macro generation





Where Allo Helps

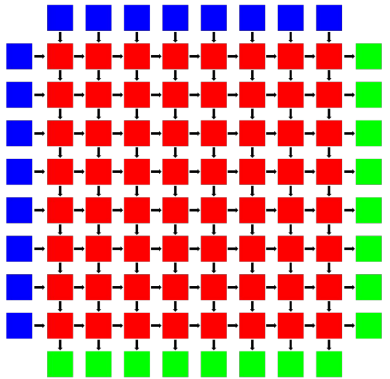
- Hierarchical flows often are manually done - user has to figure out:
  - Reuse (prove same function?)
  - How to place/tile
  - Hierarchy “cutting point”
  - Allo can do this all automatically
- After hardening macros, can’t optimize critical path
  - Allo designs have registers at macro boundaries



Extracting MLIR/HLS-Level Information

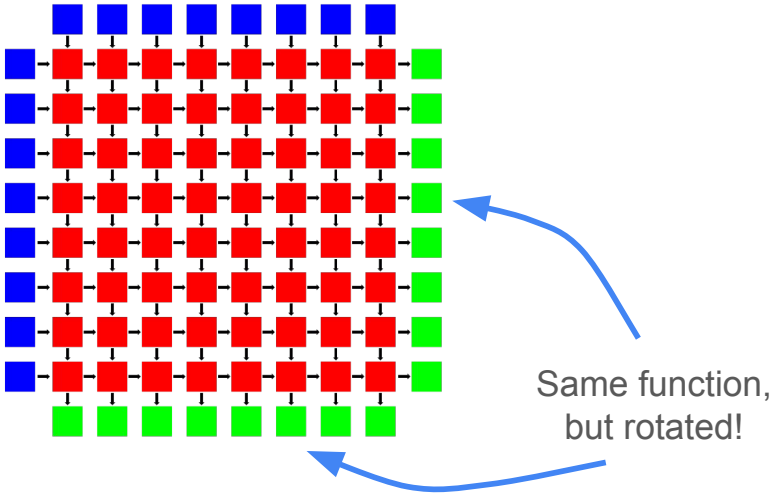
- Save spatial info lost at RTL level as MLIR attributes
  - PE IDs
  - Other specialization info
- Build graph of stream connections between PEs
  - Point to point
  - Works for all PEs, not just same-kernel PEs
- Extract more specialization info/scheduling/pragmas from HLS

```
func.func @gemm_0_1_4_4_32_32_16_0_fixed(
  %arg0: !allo.stream<i32, 8>,
  %arg1: !allo.stream<i32, 8>,
  %arg2: !allo.stream<i32, 8>,
  %arg3: !allo.stream<i32, 8>)
  attributes {
    df.kernel_name = "gemm",
    df.parent_region = "MXU_4_4_32_32_16_0",
    df.pid = "0,1",
    df.specialization_suffix = "4_4_32_32_16_0",
  }
```



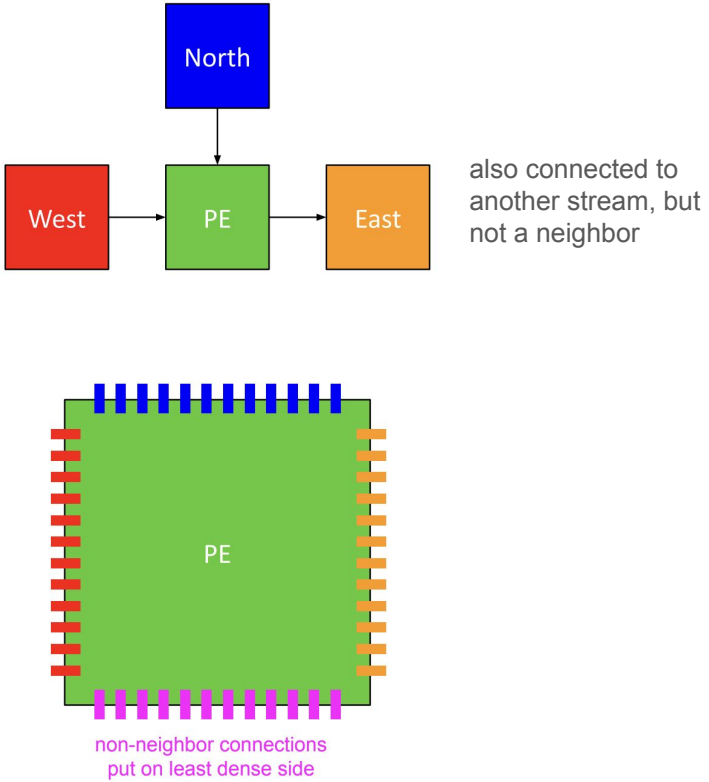
Macro Identification/Grouping

- Group PE functions/modules - figure out which are the same
  - Strip off function/variable names
  - Combine groups with D4 transformations
- End result is a group of “representative” macros to push through Stage 1 flow

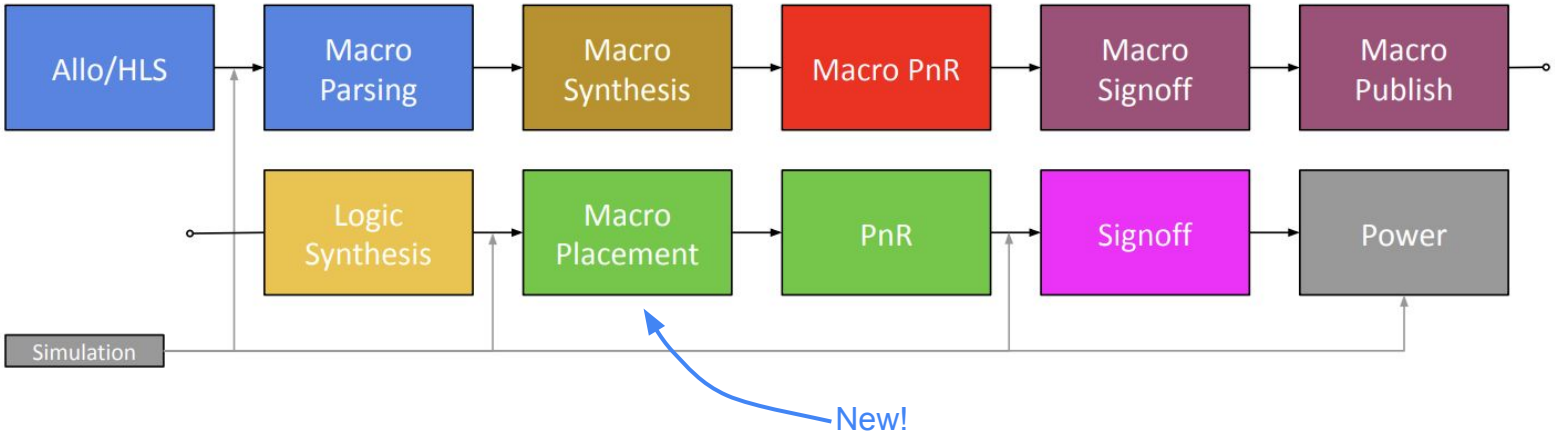


Example Downstream ASIC Task - Macro Pin Allocation

- Rich MLIR/HLS-level info can be used for downstream ASIC tasks for better performance
- Stream connection graph used to assign pins to different sides of macro
  - Allows tighter macro packing
- Pin balancing to reduce PnR routing congestion
  - Can move non-neighbor streams

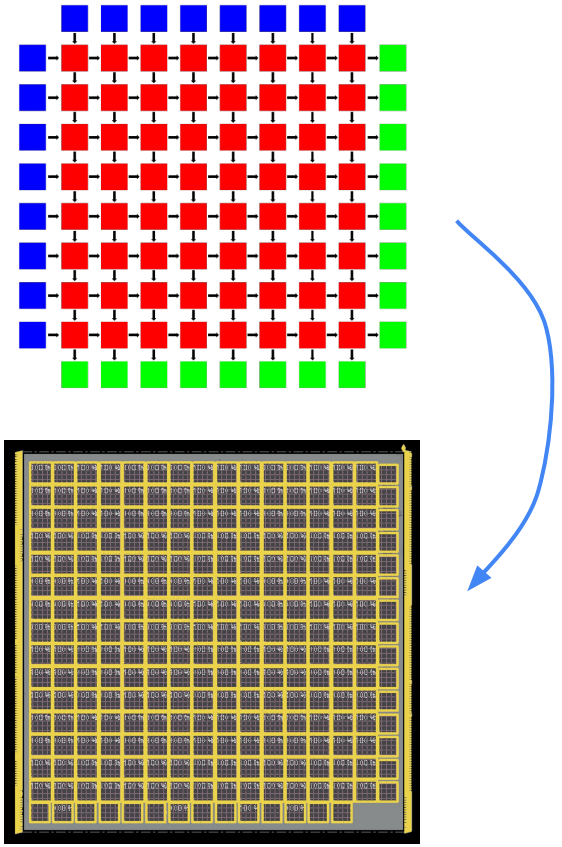


Hierarchical Flow



Macro Placement + Top Level Flow

- Now have macro physical information, can turn connection graph into physical placement
  - Obvious for same kernels, use stream connections for different kernels
- Cells in macros already optimized and routed, just route between additional cells
- Much fewer “moving parts” in optimization

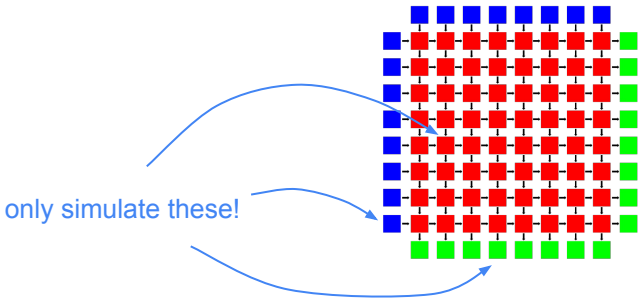


Automatic Testbench Generation + Power Simulation

- Need testbench for simulation + power
  - Have user specify inputs/outputs as for simulator build
  - Save input/output test vectors
  - Use HLS info to make SystemVerilog loader with same I/O protocol
- Power simulation: only need to simulate one macro from each group

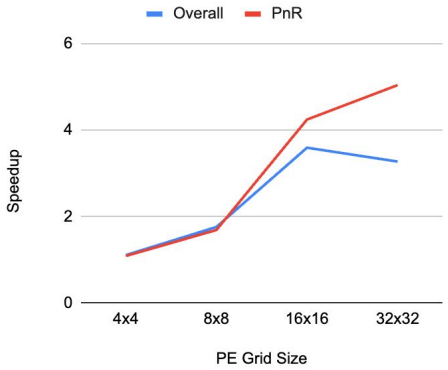
```
#pragma HLS interface m_axi port=v427 offset=slave bundle=gmem0
#pragma HLS interface m_axi port=v428 offset=slave bundle=gmem1
#pragma HLS interface m_axi port=v429 offset=slave bundle=gmem2
```

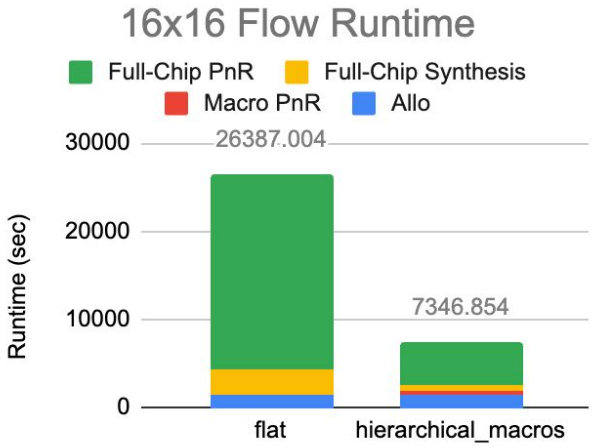
```
return {
    "top_function": "top",
    "call_signature": ["A", "B", "C"],
    "calls": [
        {
            "name": "gemm",
            "arguments": {
                "A": matrix_a,
                "B": matrix_b,
                "C": matrix_c_initial,
            },
            "expected": {
                "C": matrix_c_expected,
            },
            "comparison": {
                "C": {
                    "mode": "bit_exact",
                },
            },
        }
    ],
}
```



Runtime Results

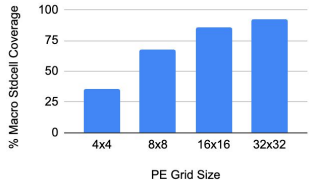
- More PEs → better speedup
- 32x32 run crashed during PnR
  - 28 hrs in (vs 5.5 hrs)
  - ~4x speedup quite significant
- Hierarchical flow saves memory
  - Peak PnR memory usage fell from 17.5 to 8 GB (16x16)
- int32 GEMM - simple PEs!

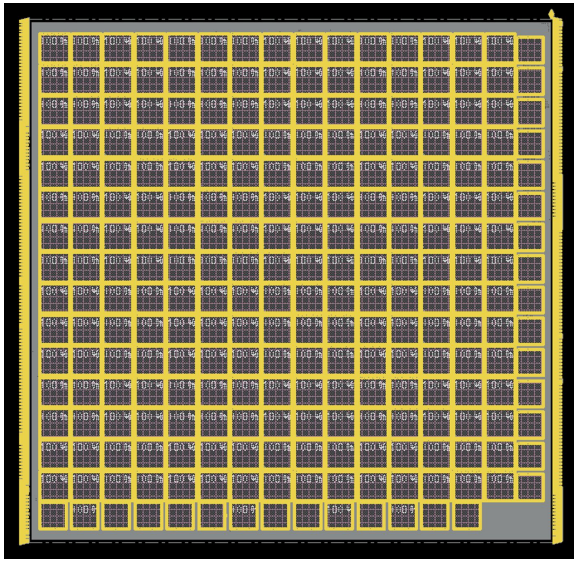




Runtime Results

- More PEs → better speedup
- 32x32 run crashed during PnR
  - 28 hrs in (vs 5.5 hrs)
  - ~4x speedup quite significant
- Hierarchical flow saves memory
  - Peak PnR memory usage fell from 17.5 to 8 GB (16x16)
- int32 GEMM - simple PEs!
- Results very preliminary

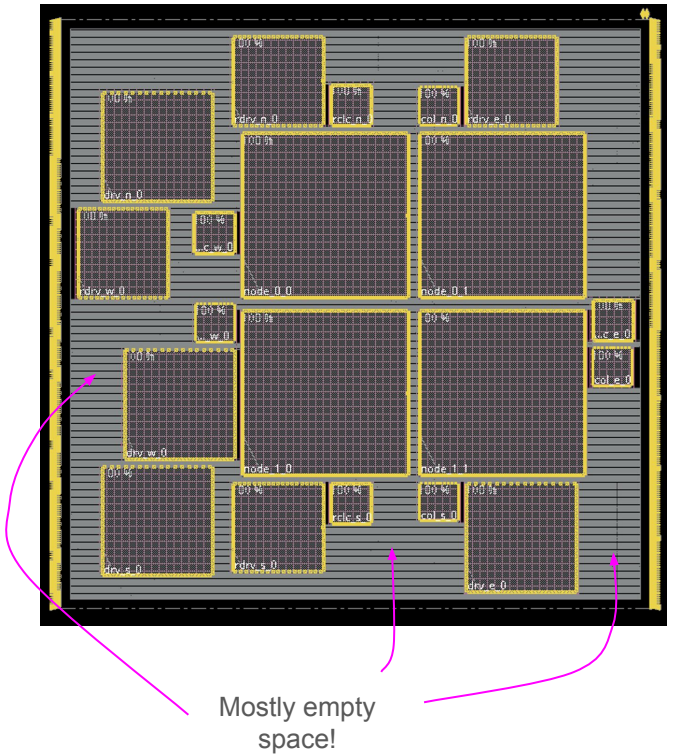




16x16: 2.1 mm (45 nm process)

EVA Rebuild Example (Thanks Zarina!)

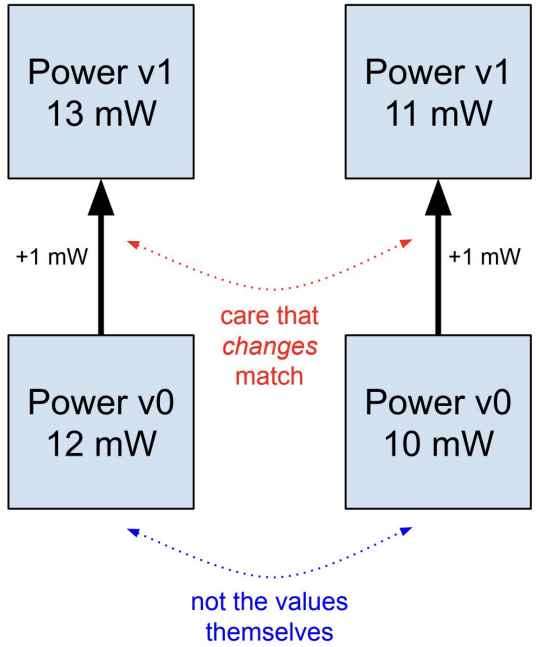
- 2x2 PE grid + drivers
- 99% of stdcells inside macros
  - Mostly empty space between
- 2.29x speedup
  - Full chip: 6.36x, good sign for scaling grid
- PPA Metrics (macro vs flat)
  - Power: 10.97 vs 10.8 mW (+1.6%)
  - Stdcell area: 2.34 vs 2.26 mm^2 (+3.7%)
  - Core area: 5.59 vs 3.21 mm^2 (+75%)
- Heterogeneous macros make macros harder to tile efficiently
  - If extra stdcells to place, can fill the gaps
  - Implies a “sweet spot” of stdcells inside vs outside macros exists



Results Takeaways

- Significant speedup
- More PEs → more speedup
- More complicated PEs → more speedup
- More heterogeneous PEs → more core area
  - If no stdcells to fill gaps

- PPA accuracy - good for power + stdcell area, but not core area
  - Do we care? Care more about how PPA changes with architectural edits than actual values



Next Steps

- Future experiments
  - Characterize speedup scaling
  - Performance stress testing
  - PPA fidelity (architectural changes)
- Support more PDKs
- Organize raw flat RTL flow

Takeaways

- Allo can now fully target ASIC (flat + macro)
- Hierarchical flow brings significant speedups while preserving key PPA metrics
  - Allo spatial geometry + reuse information crucial
  - Automatic nature allows for design space exploration - use hierarchical while developing, flat for final numbers
- MLIR + automatic hierarchical ASIC flow approach is (as far as I know) novel